# 《FPGA 入门基础》

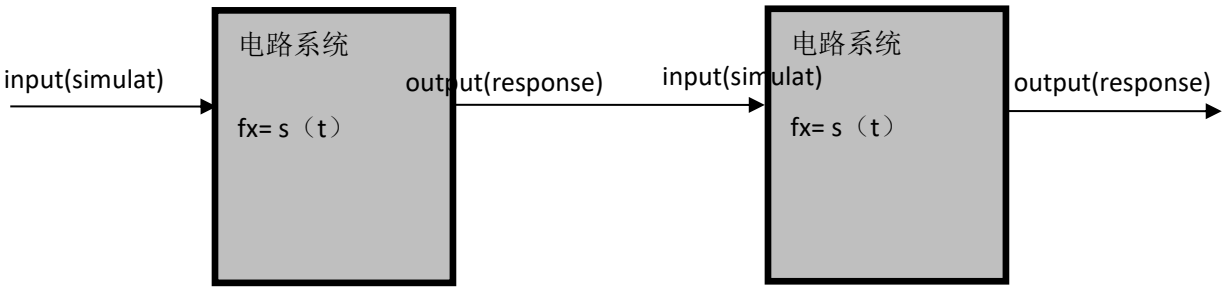
《FPGA 入门基础》 ..... 1
一 电路设计基础 ..... 3
1.1 深入理解里电路与电路设计 ..... 3
1.1.1 电路设计 ..... 3
1.1.2 电路分类 ..... 5
1.1.2 电路设计分类 ..... 6
1.1.3 EDA 技术 ..... 7
1.2 FPGA 开发基础 ..... 9
1.2.1 数字电路定义与分类 ..... 9
1.2.2 关于数字电路设计开发 ..... 12
1.2.3 FPGA 开发流程 ..... 13
1.2.4 数字电路半定制开发原理 ..... 18
1.2.5FPGA 分类 ..... 23
二：HDL 编程基础 ..... 23
2.1HDL 定义 ..... 24
2.2 HDL 的基本特性 ..... 25
.2.2.1HDL 基本特性 ..... 27
2.2.2HDL 文件类型 ..... 33
2.2.2 HDL 语言的计算机语言特性 ..... 34

# 一 电路设计基础

## 1.1 深入理解里电路与电路设计

生活中离不开电路，电路由电子元器件构成，某些电字原件构成的电路叫做电系统。

电路的本质是传递信号， 一个电路系统在控制信号的控制下，经过变换或者没有经过变化，经驱动信号的值传递给其它电路系统。




input(simulat) ： 激励信号
output(response) ： 响应

从上面可以看出，电路是由不同的电路系统构成，小的电路系统可以经过串联或者并联，也就是电路系统合并（同时也可以拆分），激励信号经过电路系统的处理输出响应信号。

电路设计的本质就是设计不同的功能的电路系统，并且简单的电路系统可以构成复杂的电路系统，所谓的功能就是信号之间值的传递方式。

### 1.1.1 电路设计

电路的本质是信号的值 传递，电路设计就是设计值的传递方式，基本电路由以下几部分构成。

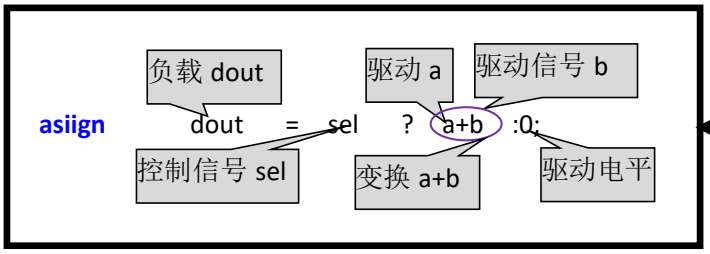
驱动：　　传递值的电路为驱动，传递值的信号叫做驱动信号

负载：　　接收传递值的电路为负载，接收传递值的信号叫做负载信号

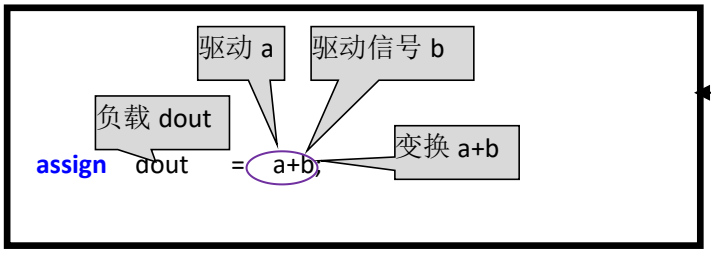
变换：　　我们把值在传递过程中的运算电路叫做变幻，变幻需要**根据实际需求设计是否需要变换电路**。

控制：　　信号值在传递过程中如果需要控制，根据**实际需要设计控制电路**，对应的信号叫做控制信号。

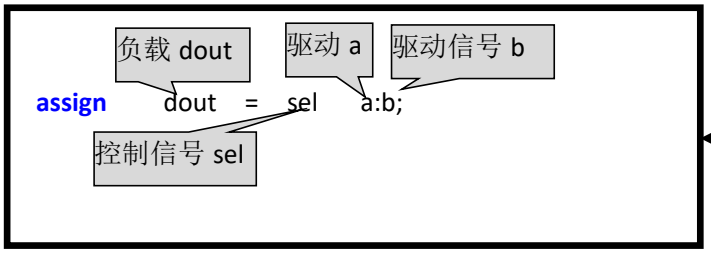
如：　电路设计 4 种方式



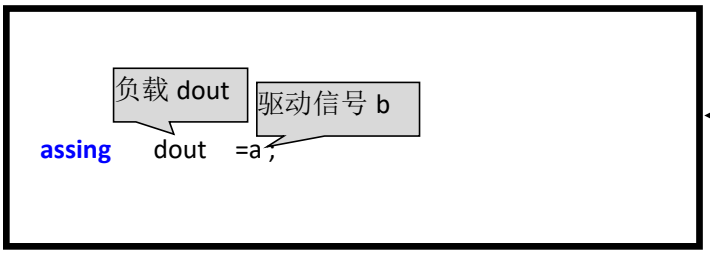

驱动信号：a,b
负载信号:dout
控制信号：sel
变换：a+b




驱动信号：a,b
负载信号:dout
变换：a+b




驱动信号：a,b
负载信号:dout
控制信号：sel




驱动信号：a,b
负载信号:dout

在电路设计中，信号在传递过程中参与的信号主要包括驱动信号，负载信号，控制信号等，这路需要注意一个很重要的概念，我们通常把驱动信号和控制信号组成的信号叫做敏感信号。

敏感信号在数字电路设计中尤为重要，在 HDL 语言如果没有标准全敏感，有时电路会出错。

如：

完整的敏感信号

```
always   @( sel,a,b     )
            if(sel )
               dout   =a +b;
            else
              dout   =0;
```

完整的敏感信号

```
always   @( * )
            if(sel )
               dout   =a
+b;
            else
              dout   =0;
```

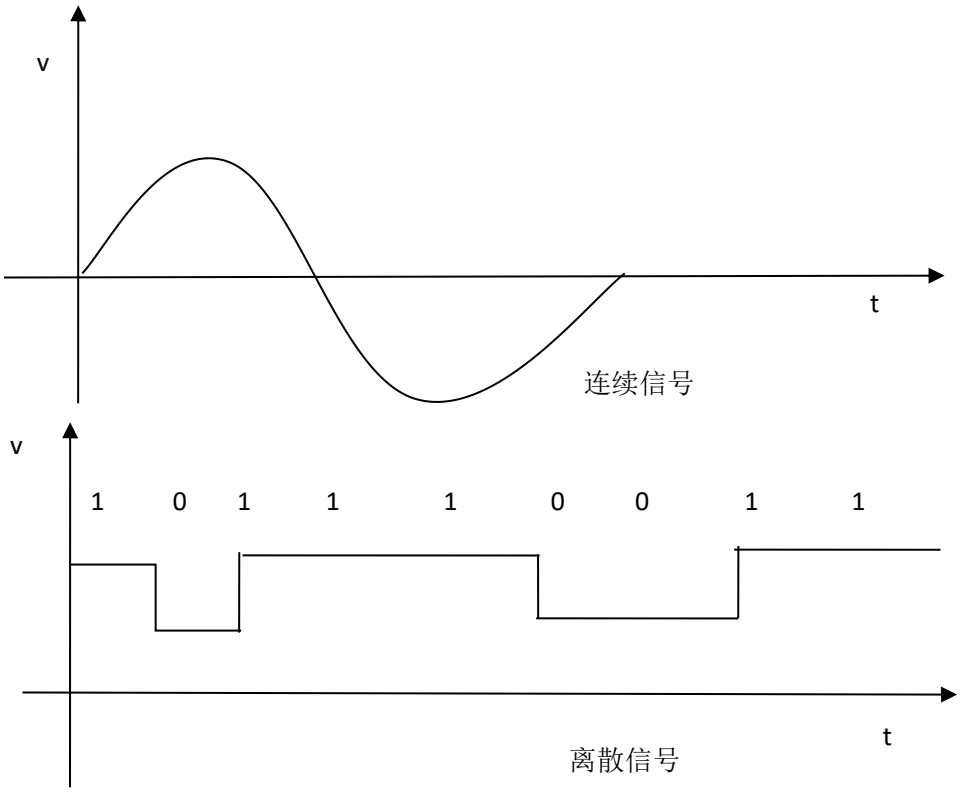
整的敏感信号不完整

```
always   @( a,b     )
            if(sel )
               dout   =a +b;
            else
              dout   =0;
```

## 1.1.2 电路分类

电路本质是传递信号，通常我们按照所传递信号特征进行分类，在设计中电路分为如下 2 个标准。

| 信号类型 | 电路类型 | 描述 |
|---|---|---|
| 连续信号 | 模拟电路 | 在时间轴上，传递的是连续信号，它由电子元件构成 |
| 离散信号 | 数字电路 | 在时间轴上，传递是离散信号，　它由电子元件构成 |




以上是从设计角度统一划分，也可以按照功能划分，比如 GPU，CPU，DSP...等属于特定功能的电路。

## 1.1.2 电路设计分类

电路设计属于电子开发当前从应用上来看，电路设计包含 3 种常规的设计：

模拟电路设计　　：聚焦模拟信号的电路设计
数字电路设计　　：聚焦数字信号的电路设计
数字模拟混合 PCB 设计　：聚焦混合数字模拟全功能的电路设计

以上是电路设计的分类，属于电子开发，电子开发除了电路设计之外，常见的还有嵌入式开发，软件设计。

### 1.1.3 EDA 技术

电路设计如何实现是我们关注的问题，传统的方法就是绘制电路图，这种设计方法对于复杂的超大规模电路设计非常不友好，随着计算机的出现，通过计算机软件进行设计成为现代电路的主流方法，理想的是我们告诉软件我们需要什么样的电路，右软件辅助设计电路，这种设计方法我们叫做 EDA 技术。

EDA（电子设计自动化），是通过计算软件辅助设计，所有辅助软件我们叫做 EDA 软件，是现代电路设计的必然，因此对于合格的电路设计师必须掌握相关的 EDA 工具。

对于数字电路设计，EDA 工具大概可分 3 种：

1： 集成开发环境

集成开发环境就是通过可视化软件界面，或者命令窗口，统一管理与开发有关的 EDA 工具，以及其他辅助操作。

对于可视化软件界面本质还是软件，但是不是具备单一功能的辅助 EDA 工具，而是一个管理软件。

通常通过集成开发环境可以完成几乎所有有关的电路设计开发任务，不同开发任务的集成开发环境不同，这里我们重点讨论 FPGA，对于 FPGA 集成开发环境与厂商有关。

| FPGA 厂商 | 集成开发环境 | 备注 |
|---|---|---|
| intel FPGA | Quartus Prime Design Suite | 全球第 2 大 sram fpga 供应商 |
| AMD FPGA | Vivado ML Design Suite | 全球第 1 大 sram fpga 供应商<br>基于最新系统的支持，目前 7 系列 |
| | ISE Design Suite | 全球第 1 大 sram fpga 供应商<br>基于全系列的支持。 |
| ACTEL | Libero | 全球 flash fpga 的供应商 |
| lattice | Diamond | 全球反熔丝 fpga 的供应商 |
| 安路科技 | TD | 国产第 2 大 FPGa 供应商 |
| 紫光同创 | PDS | 国产第 1 大 FPGa 供应商 |

所有的 fpag 以 AMD xilinx 为向导，通常我们必须掌握 Vivado ML Design Suite,Quartus Prime Design Suite。

2： 独立辅助软件

电路设计中，独立辅助软件是单一功能的 EDA 工具，比如 MODELSIM。当前设计中比较常用的为 **modelsim**。

以下列举出 FPGA 开发相关的常见的独立辅助软件。

| EDA 软件 | 软件名 | 备注 |
| --- | --- | --- |
| 仿真工具 | ModelSim | RTL 级仿真 |
| 验证工具 | QuestaSim | 验证工具，支出 UVM |
| 综合工具 | Synplify | 高级综合器 |

对于普通 FPGA 开发者需掌握 **Modelsim**，对于高级开发者可能需要熟悉 Synplify，对于 IC 设计可以了解 QuestaSim。

3：文本编辑器

是计算机中比不可少的工具，编程必备工具，对于电路设计来说涉及到编程。

对于程序员，文本编辑器必不可少，选择一个好的文本编辑器非常重要，通常集成开发环境中都集成了文本编辑器工具，但是都比较简易，因此对于一个经常开发的人员来说，需要一个单独的文本编辑器，对于 FPGA 推荐 **Notepa ++** .

以下列举出，我们需要安装的 EDA 软件：

| EDA 软件 | 下载链接 | 备注 |
| --- | --- | --- |
| Quartus Prime Design Suite | 链接：https://pan.baidu.com/s/1flNRSrzBX7yvmj1lgVIX1Q?pwd=c06e<br>提取码: c06e | Quartus 安装过程中自带 modelsim，所以不需要安装 |

| | | modelsim |
|---|---|---|
| Vivado ML Design Suite | 通过网盘分享的文件：2018.3.zip<br>链接：https://pan.baidu.com/s/1OCHYh5uZd3BNA1wgXEEVug?pwd=aebd 提取码: aebd<br>--来自百度网盘超级会员 v6 的分享 | |
| ModelSim | 链接：https://pan.baidu.com/s/1flNRSrzBX7yvmj1lgVIX1Q?pwd=c06e<br>提取码: c06e | |
| notepa++ | https://cn-notepadplusplus.com/downloads/ | 文本编辑器 |

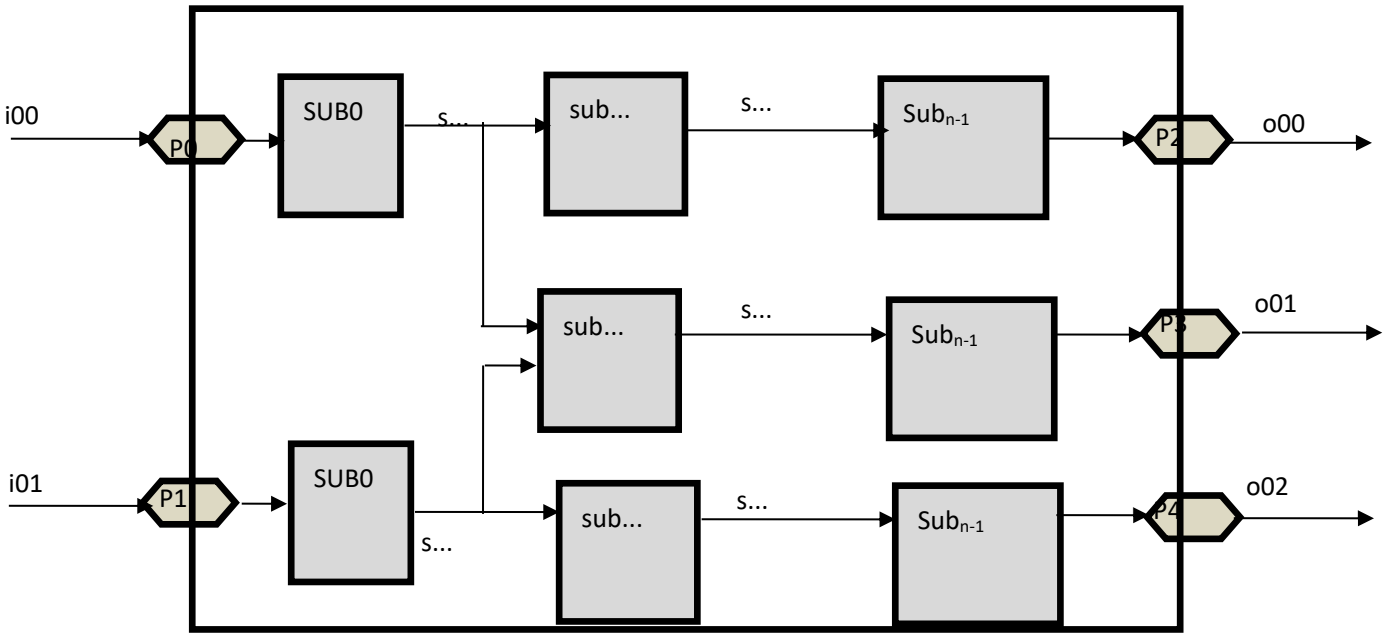
## 1.2 FPGA 开发基础

数字电路设计开发是一整套操作，对应开发者来说必须熟练掌握开发原理与开发流程，并且开发流程与原理都是都是固定的。

### 1.2.1 数字电路定义与分类

■ 数字电路定义

定义：由电子元件构成，用来传递数字信号（离散信号），具备一定功能的电路系统。



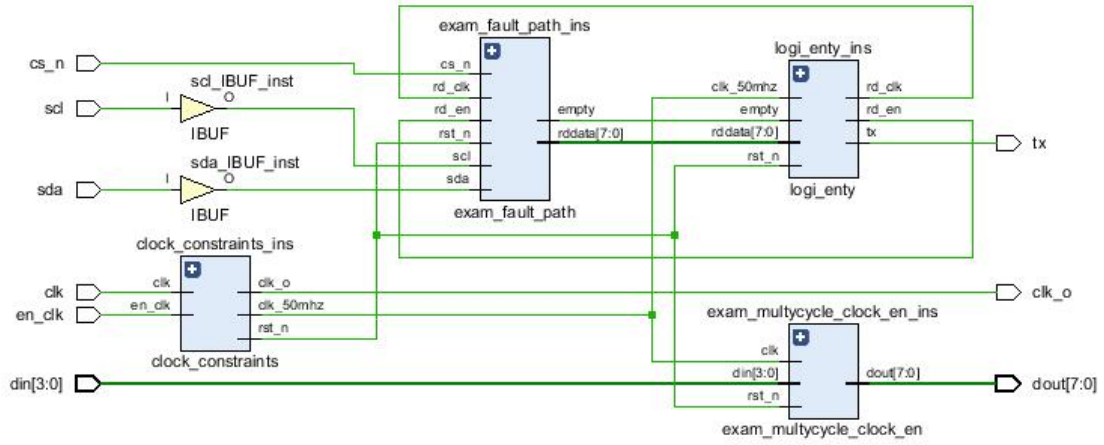

标准的数字电路结构示意图

p0,p1,p2,p3,p4 :端口
IOO,IO1,OOO,O01,O02 : 端口信号
s... :子电路的驱动或者负载，以及控制信号
sub0,$sub_{n-1}$,sub... :子电路系统

一个标准的数字电路构成如下：

端口 ：用于接收或者输出外部信号，是一个完整电路系统的组成部分,由于端口本质还是用来传递信号，在实际中端口有 4 种

| 端口类型 | 描述 |
|---|---|
| **input** | 表示该端口用来接收外部信号，通常需要接一个 ibuf |
| **output** | 表示该端口输出外部信号， 通常需要接一个 obuf |
| **buf** | 表示该端口为反馈输出，为 output 的子类 |
| **inout** | 表示该端口即可以输出信号，也可以接收信号，通常需要接一个 tri_buf（三态门），输入或者输出通过 tri_buf 控制 |




子电路 ：对于一个电路系统被分割的小的电路系统，对应我们电路中的子模块。

信号 ：电路的血脉，数据传输的管道。

**NOte:**在实际电路设计中，任何电路系统（包括子系统）通常用模块表示。

■ 数字电路分类

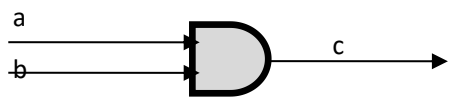
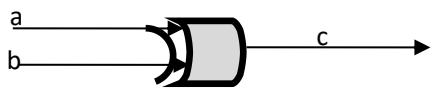
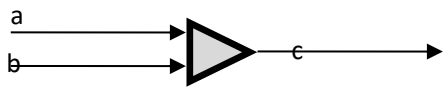
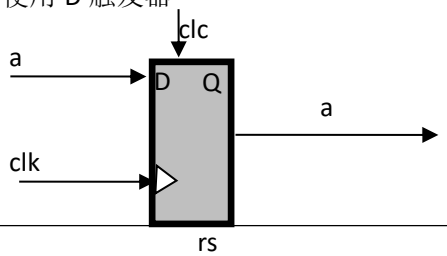
在数字电路设计中，要明确设计电路类型，根据实际情况选择电路设计类型

| 分类标准 | 电路类型 | 定义 |
|---|---|---|
| 未采样 | 组合电路 | 由与门，或门，非门构成的传递数字信号的电路 |
| 采样 | 时序电路 | 对组合电路的值进行采样，并且将寄存器保存到寄存器中。<br>Note:电路中**禁用锁存器** |

组合电路：是数字电路设计的基础，**任何组合电路都可以由与门，或门，非门构成**。

时序电路：依赖于组合电路，只是对组合电路值进行采样寄存，本质还是以组合电路为基础。

总结：通过分析可以看出数字电路设计的关键解决组合电路的设计，设计的基本元素为与门，或门，非门，触发器。

| 数字电路元件 | 通用符号 |
|---|---|
| 与门 |  |
| 或门 |  |
| 非门 |  |
| 触发器 | 所有 FPGA 使用 D 触发器<br>  |

| | ↑ |
|---|---|

## 1.2.2 关于数字电路设计开发

数字电路设计开发，在实际中分为前端和后端，并且衍生出主流的开发方法有 2 种。

前端：通俗点讲通过 EDA 技术设计电路
后端：通过生产链进行流片。

1：全定制开发（ASIC）
全定制设计 = 前端 + 后端

| 开发类别 | 开发者 | 描述 |
|---|---|---|
| 前端 | 设计者 | 设计者设计电路 |
| 后端 | 设计者+代工厂 | 设计者委托代工。<br>**后端完全依赖于前端设计。** |

2：半定制（可编程逻辑器件开发）

半定制 $_{1/2}$ = 前端
半定制 $_{1/2}$ = 后端

| 开发类别 | 开发者 | 描述 |
|---|---|---|
| 前端 | 前端设计者<br>如：用户 | 负责编程器件的程序开发<br>**前端设计依赖于后端设计。** |
| 后端 | 后端设计者+代工<br>如：amd fpga xilinx<br>intel FPGA ALTERA<br>紫光<br>安路科技 | 负责编程器件的开发 |

3:半定制和全定制开发设计特点
当我们进行数字电路设计的时候，首先就是棉铃开发方式的选择，根据各自的特点选择合适的开发方式。

| 设计方法 | 特点 |
|---|---|
| 全定制 | 优点<br>1：用户自主性更高<br>2：电路性能更好 |

| | |
|---|---|
| | 3：功耗，速度等级更高<br>缺点<br>1：大部分电路需要自主开发<br>2：时序要更高<br>3：人力成本更高<br>4：开发环境以 Linunx 操作系统为主<br>总结：当我们需要批量功能相同电路的时候，对设计周期容忍度高，无疑是最好的选择 |
| 全定制 | 优点<br>1：电路灵活度非常高、试错成本非常低<br>2：电路升级灵活度高<br>3：开发环境为 windows 为主<br>4：很多复杂功能不需要我们单独开发<br>缺点<br>1：性能相对于 ASIC 较弱<br>2：功耗，速度相对于 ASIC 较低<br>3：后端芯片成本高，人力成本低<br>总结：如果我们电路需要定期升级，需求不断变换，用量少的情况下，这种开发无疑是最好的选择。 |

### 1.2.3 FPGA 开发流程

数字电路开发都遵循一定的标准的开发流程，不同的开发类型开发流程有一定个差异化，这里重点介绍 FPGA 开发流程。

FPGA 标准的开发流程分为前后 2 部分：

开发流程前部分：

第 1 步：需求分析

提取电路设计的相关要求和参数，必须明确我们要设计的目标电路。

第 2 步：调研与可行性分析

针对目标电路进行调研，通过互联网等手段查找相关资料，并研究，初步凝练出一套设计方案，并分析是否可行，并且判断是否需要修需求。

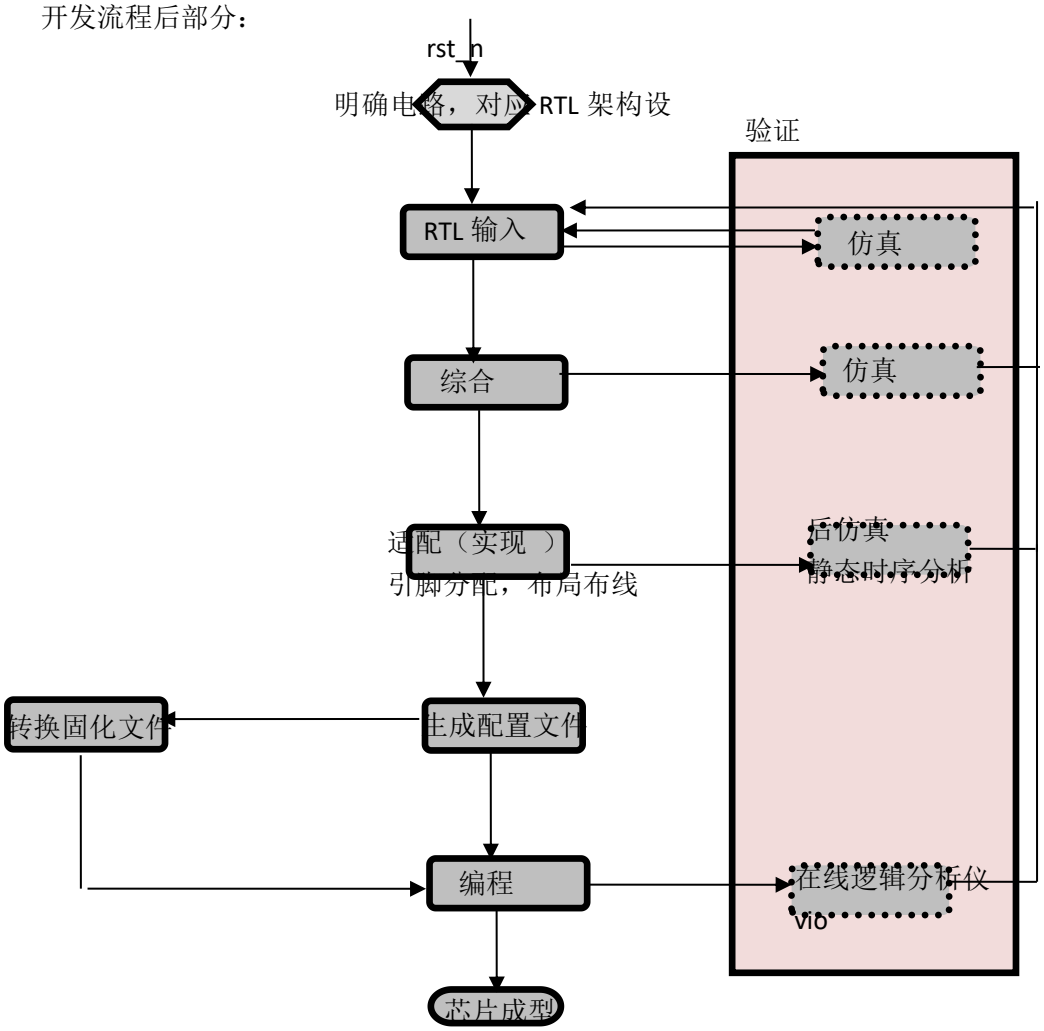
第 3 步：算法与系统设计

如果电路设计中涉及到算法，我们需要进行算法验证，与 **RTL 算法转换**。如果需要系统架构的设计，则设计系统架构。

第 4 步：RTL 架构设计

设计出详细的电路架构，后续编码等操作必须完全符合当前架构的设计。

开发流程后部分：




开发流程设计案例

标题：　　LED 点灯

要求：

1：4 个 led 灯

2: 间隔 1 亮 1 灭

| led 编号 | 状态 |
|---|---|

| | |
|---|---|
| 0 | 亮 |
| 1 | 灭 |
| 2 | 亮 |
| 3 | 灭 |

1：需求分析

- led 灯点亮
- 4 个 led 灯
- 间隔 1 亮 1 灭

2:调研与可行性分析

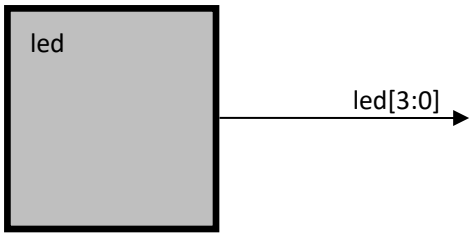
led 为高电平对应的 3.3v，点亮，我们通过信号 led[3:0]分别控制 4 个灯，控制如下：

| led[3:0] | led 编号 | 状态 | 值 |
|---|---|---|---|
| led[0] | 0 | 亮 | 1 |
| led1] | 1 | 灭 | 0 |
| led[2] | 2 | 亮 | 1 |
| led[3] | 3 | 灭 | 0 |

3:算法与系统设计

| 项目 | 备注 |
|---|---|
| 算法 | 没有设计到算法 |
| 系统设计 | 需要系统设计 |

4:RTI 架构设计




5:集成开发环境中开发

设计流程后部分需要结合集成开发环境,大部分的操作通过 EDA 工具自动完成，比如综合，布局布线，生成配置文件。

■ 开发过程中涉及到的相关概念

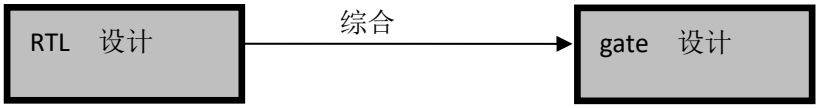
**RTL（Register Transfer Level） ：**

在数字单路设计过程，设计具有抽象级别，典型的层次为，系统级，算法级，RTL 级，门级，其中 rtl 级是电路设计中非常关键的一个层级。

RTL 设计通常为符号文件，HDL 语言设计文件等，在 FPGA 开发流程中，所谓的 RTL 输入就是编写符号文件或者 HDL 语言程序。

**综合 ：**

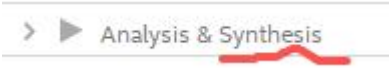
当我们设计的电路设 rtl 级设计的时候，电路设计还未到最最低层次，理论上行 最低层次的电路设计才算真正电路设计，实际中需要借助计算机辅助技术（EDA）工具，即综合器，把 RTL 设计转换为门级设计。




以上是关于综合的传统定义，在现代数字电路设计关于综合的概念已经超出了传统的定义，赋予其更广泛的含义，现代定义为高层次向低层次的转换都可以理解为综合。

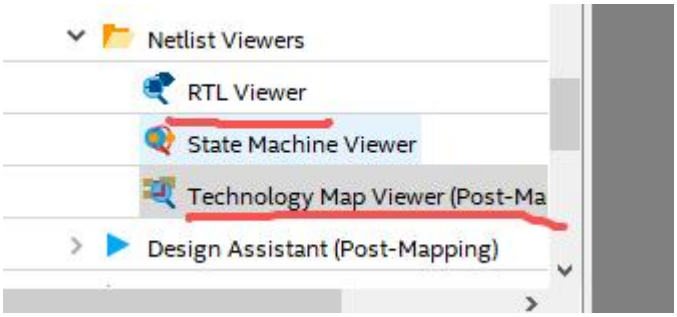
比如：xilinx 推出的高层次综合器（HLS-High-Level Synthesis）
hls 是将算法级设计转换 RTL 级设计的综合器，

在开发流程所谓的综合就是通过集成开发环境调用综合器，完成 rtl 级到门级的转换，并且是综合器自动完成，一般是后台



## 综合分析：

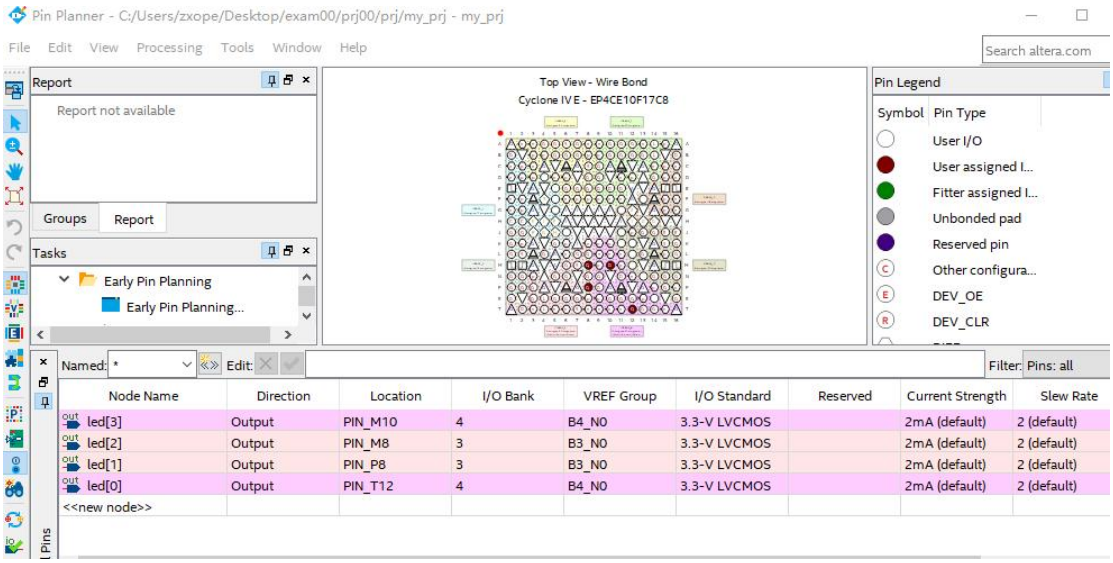
针对当前门级电路进行分析，通常在集成开发环境中我们直接看电路图（**RTL 级电路图/gate 级电路图**），




## 适配：

对于半定制电路来说，前端设计的电路与后端设计的芯片之间的资源分配，涉及到 3 方面的资源分配。

引脚分配：将我们设计的 top_leve entity 端口与后端芯片阴间分配与绑定，在集成开发环境中需要通过阴间分配公工具手动根据自己的电路板进行引脚分配.




布局： 前端电路设计中门电路与 clb 等资源的分配，一般 implement 工具自动完成，也支持手动，但是只是作为辅助。

布线 ： 前端电路设计中门电路与线资源的分配，一般 implement 工具自动完成，也支持手动，目前不支持手动。

**配置文件：**

配置文件是半定制开发非常重要的文件，是前端设计和后端设计之间无缝衔接的桥梁，用来配置后端芯片，是前端设计在**最终文件**。配置文件为二进制文件，不同的集成开发环境中文件名不一样.

| 集成开发环境 | 配置文件 |
|---|---|
| QUARTUSS II | *.sof |
| VIVADO | *.bit |

**固化文件：**

固化文件在半定制开发中某些**掉电失去配置属性编程器件**（sram　fpga ），需要将配置文件存放在 flash 中，上电自动加载,这就需要将配置文件转化为 flash 存储文件，叫做固化文件。

**编程：**

半定制开发的主要特性就是编程，将配置文件已下载（手动操作）的方式，或者固化文件以加载（自动加载）的方式发送给内部编程单元自动完成电路构建的过程。

**仿真：**

仿真就是模拟信号之间值的时序变化，以便于及时发现电路设计中时序问题的一种手段，需要借助仿真器（如：modelsim）.

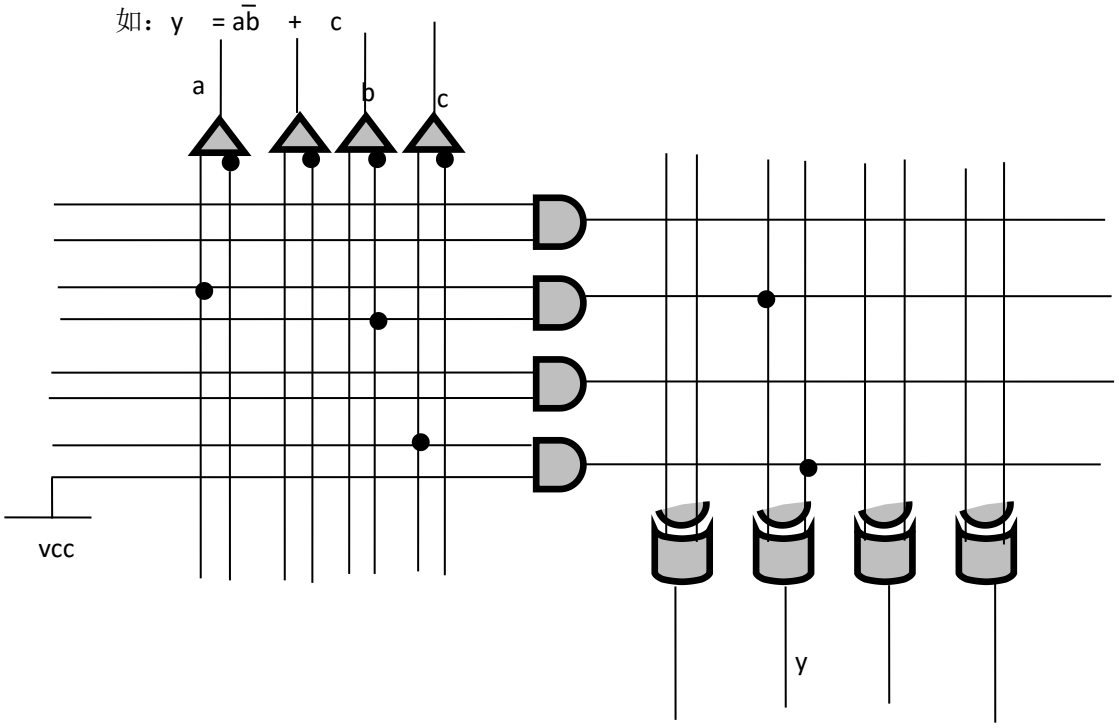
**在线调试：**

所谓在线调试借助一些特殊电路，比如嵌入式逻辑分析仪，VIO 等工具帮我们监测内部个信号之间值时序变化的手段。

## 1.2.4 数字电路半定制开发原理

在数字电路中组合电路是关键，我们也知道组合电路有 2 种表示方式，分别是逻辑表达式和真值表。

■ 逻辑表达式：用来表示信号之间逻辑关系的一种方式，通过与门，或门，非门表示。在《数字电路》这门专业课的课本中我们可以看出任何组合电路都由与门，或门，非门构成，换句话说就是任何组合电路的底层电子元件为与门，或门非门。




通过以上分析，如果我门的后端芯片中有与门或者排列的阵列，并且具备可编程属性，就可以按照以上原理构建各种逻辑表达式，基于与门或门阵列的可编程器件叫做 CPLD。

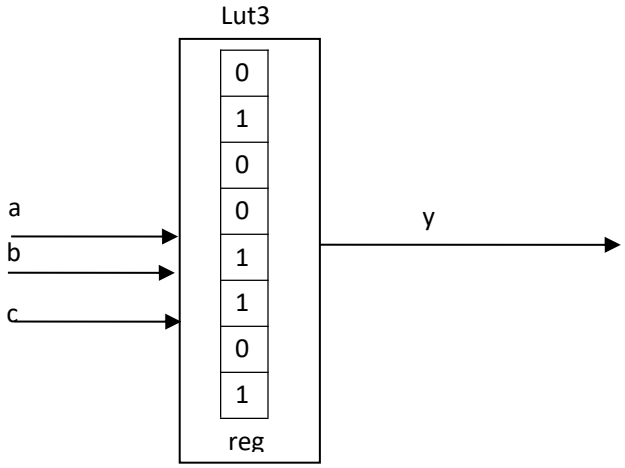
■ 真值表：将输入信号与输出信号之间值的变化的关系用一个表格记录，把这个表格叫做真值表。

如：y = $\overline{ab}$ + c 对应的真值表如下：

| a | b | c | y |
|---|---|---|---|
| 0 | 0 | 0 | 0 |

| | | | |
|---|---|---|---|
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

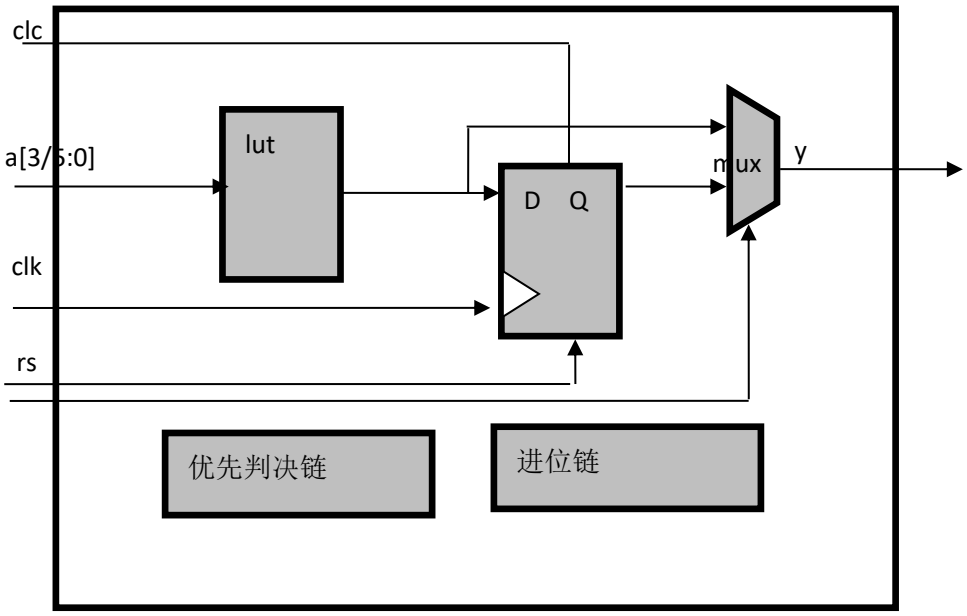
真值表本质上就是 LUT(Look-Up Table),相当与存储器，表示如下：




lut 与真值表示意图

通过以上可以看出通过 lut 同样可以实现组合电路，通过小的 lut 合并可以构建复杂的组合电路，如果后端芯片通过编程合并 lut，就可以实现不同组合电路的实现，我们把基于 lut 的编程器件叫做 FPGA。

对于现代 fpga 而言，基本都是基于 4 输入或者 6 输入的 lut，但是最小的逻辑编程单元是 CLB，通常一个 CLB 种含有 1 个 lut,有的也包含 2 个或者 2 个以上，CLB 的结构如下：




FPGA clb 构成示意图

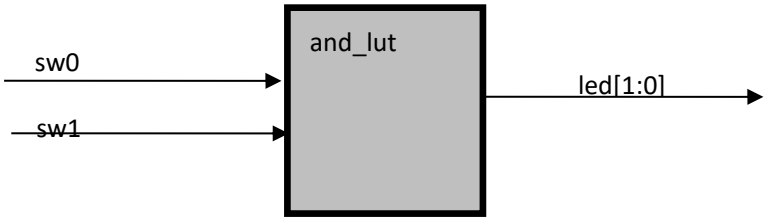
总结：所有半定制开发后端芯片都必须具备可编程属性，通常为以上 3 中编程方法。

■ 半定制开发原来设计案例

标题： 通过 lut 构建与门
要求：
1：采用 2 输入 lut,即 lut2
2:通过 2 个按键作为 lut 输入的触发信号
3：与门输出用 led 灯作为观测信号

| {w0,w1} | led | led 亮灭（默认高电平亮） |
|---|---|---|
| 00 | 0 &0 = 0 | 灭 |
| 01 | 0 &1 = 0 | 灭 |
| 10 | 1 &0 = 0 | 灭 |
| 11 | 1 &1 = 1 | 亮 |

RTL 架构设计




端口说明

| 端口信号名 | 描述 |
|---|---|
| 与门输入 | |
| sw0 | 与门输入信号 sw0，利用按键触发，按键状态如下：<br>0：按下<br>1：未被按下 |
| sw1 | 与门输入信号 sw1，利用按键触发，按键状态如下：<br>0：按下<br>1：未被按下 |
| 与门输出 | |
| led | 与门输出，利用 led 检测结果是够正确，默认低电平亮<br>只要有任何一个按键按下，led 都是亮<br><br>led[1]:检测 lut2 计算结果<br>led[0]:检测综合计算结果 |

与门真值表（lut）

y=a&b

| a | b | y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

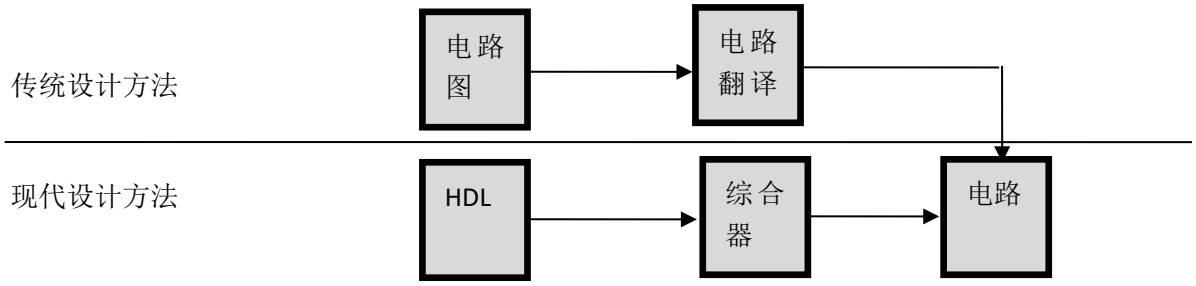
## 1.2.5FPGA 分类

FPGA 作为半定制可编程器件（后端设计芯片），由于工艺特性不同，目前按照工艺制作类型有 3 种。

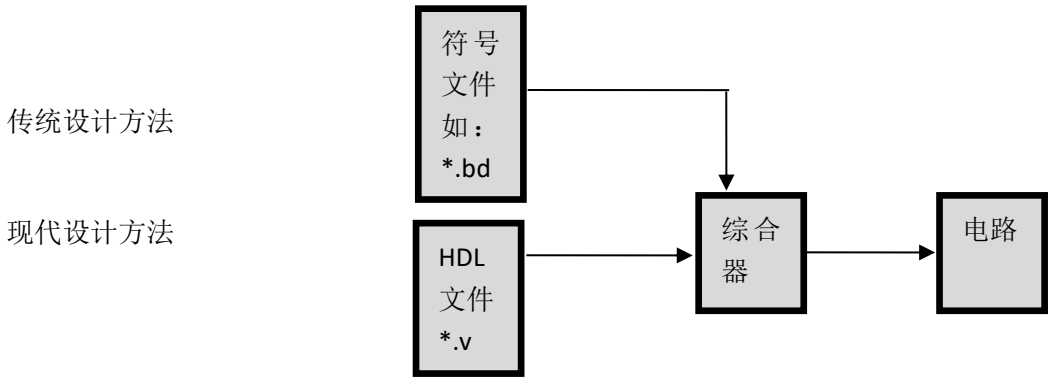
| FPGA 类型 | 全球最大的后端设计公司 | 工艺特点 |
| --- | --- | --- |
| SRAM FPGA | AMD FPGA（xilnx ） | 每次上电需要重复编程配置<br>为了避免重复编程，芯片通常必须支持自动加载<br><br>**三者中抗高温，高压能力最弱，适合自然环境。** |
| Flash fpga | Lattice | 每次上电不需要重复编程和配置，但是有编程次数限制。<br><br>**三者中抗高温，高压能力折中，适合高温，高压环境** |
| 反熔丝 | Actel | 只能编程一次<br><br>三者中抗高温，高压能力最强，但是成本太高 |

# 二：HDL 编程基础

数字电路设计传统的设计的设计方法是绘制电路图，但是存在非常大的局限性，不适合用来设计超大规模的集成电路，随着 EDA 计数的发展，新型的设计方法出现。



现在的电路设计任然保留了传统设计，二者可以结合使用，唯一不同是，2 二设计已经容易为一体，主流以 hdl 为核心。




## 2.1HDL 定义

HDL 定义：叫做硬件设计语言，用于现代数字电路设计，是数字电路设计的工具，与综合对方的唯一方式。

HDL 本质：HDL 本身为计算机语言，是最底层的语言，用来**描述电路的设计需求**。
编写 HDL 代码本质就是告诉综合器我需要什么样的电路，同样的程序，不同综合器综合处的电路也不一样。

写好 HDL 程序从以下几个方面入手：

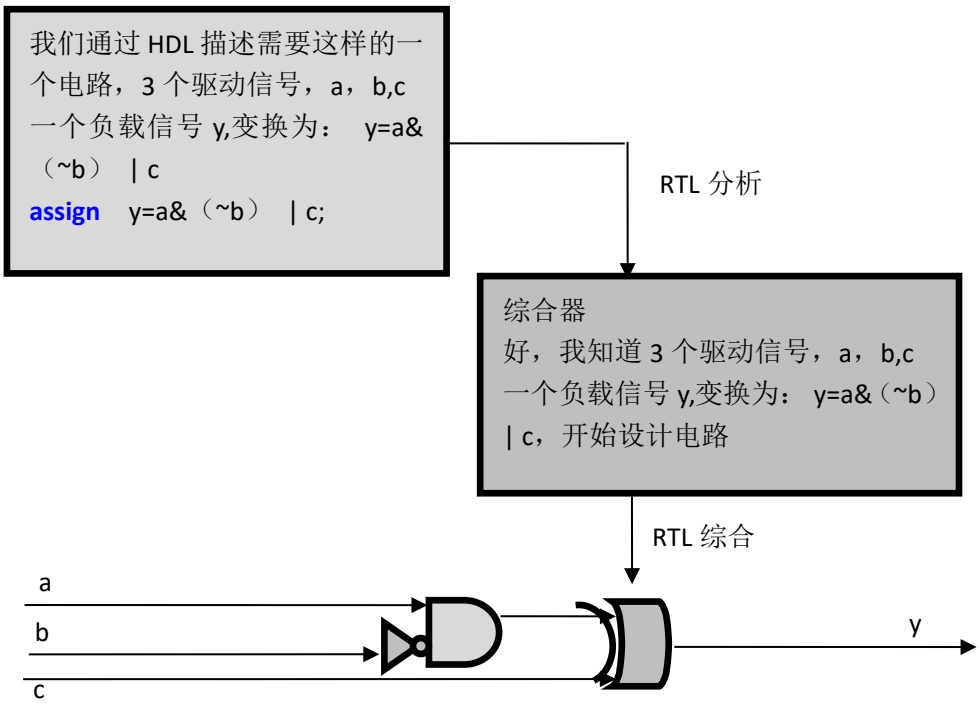
1：逻辑关系明确
2: 简，没有冗余
3： 编码规范

hdl 与纯计算机语言相比， 最显著区别程序语句不是串行的，都是并行的。
如：

通过分析可以知道并非真正的设计电路，而是描述电路需求。

如：$y=\overline{ab} + c$

对于上述逻辑表达式，代表是一个组合电路，我们通过 HDL 语言描述这个电路样子。




目前主流的 HDL 语言有 3 个，分别是 verilog hdl ,vhd ,ahdl ,国内 80%开发者都使用 verilog hdl（简称 verilog）,通常地我们的要求精通与门，熟悉其他语言。

## 2.2 HDL 的基本特性

HDL 语言也属于计算机语言，所以具备普通计算机语言的特点，同时还具备 HDL 的特性。

| 特性 | 特点 |
| --- | --- |
| 普通计算机语言特性 | 1：变量<br>2：数据类型<br>3：运算<br>4：函数<br>5：文件操作<br>6：头文件 |
| HDL 特性 | 1：模块化设计<br>2：端口<br>3：信号<br>4：电路运行特征<br>5：电路可配置 |

我们熟悉的 C 语言，JAVA 等语言，只有普通计算机语言的特点，没有 HDL 特性，HDL 语言是二者特性的统一体，我们要熟练掌握 HDL 语言，我么就需要从这 2 个方面入手。

为了更好的尽快掌握 HDL，我们介绍语言的基本构造，
人类语言的的特点是：字 -> 词 ->句 -> 章
机器语言的特点 ： 字 -> 句 -> 程序

■ 字

关键字 ：是构成句的必要部分，具备一定的语言，比如 assign
信号名 ：信号的名字， 如：**reg** [7:0] a；
变量名 ： 变量的名字 如：**real** a;

■ 句

是构成程序的基本单位，所有的程序都必须是完整的句构成，如：**assign** y =a;

■ 程序

有多条不同语句构成，用来完成我们的设计

## .2.2.1HDL 基本特性

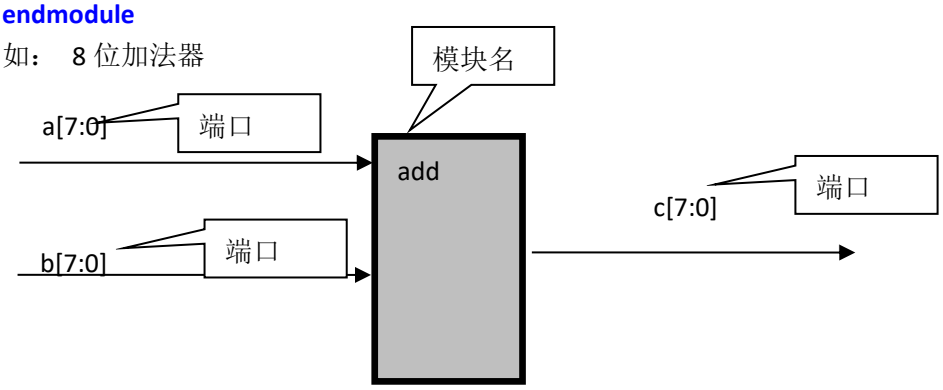
■ 模块

定义：HDL 中用来表示一个**独立的电路系统**，模块可以合并，也可以拆分。

**module** 模块名

**endmodule**

如： 8 位加法器




---

verilog 描述电路模块如下：

**module** add

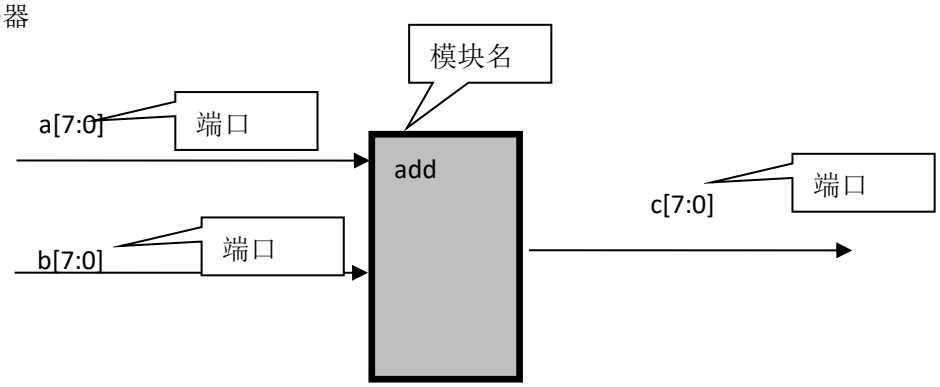
**endmodule**

■ 端口

定义：电路的基本属性，用于个模块之间传递信息的通道，任何电路都有该属性，端口通过信号传递信息，并且一个端口可以通过多个信号传递信息。

| 端口类型 | 描述 |
|---|---|
| **input** | 表示该端口用来接收外部信号，通常需要接一个 ibuf |
| **output** | 表示该端口输出外部信号， 通常需要接一个 obuf |
| **buf** | 表示该端口为反馈输出，为 output 的子类 |
| **inout** | 表示该端口即可以输出信号，也可以接收信号，通常需要接一个 tri_buf（三态门），输入或者输出通过 tri_buf 控制 |

如： 8 位加法器




---

verilog 描述如下：

```
module      add

(
            input     [7:0]     a,
            input     [7:0]     b,
           output     [7:0]     c
);

endmodule
```

解释：该描述为我需要个叫做 add 的电路模块，端口定义如下：

| 端口 | 端口类型 | 描述 |
|---|---|---|
| a[7:0] | **input** | 加法信号 a |
| b[7:0] | **input** | 加法信号 b |
| **c[7:0]** | **output** | 加发结果输出 |

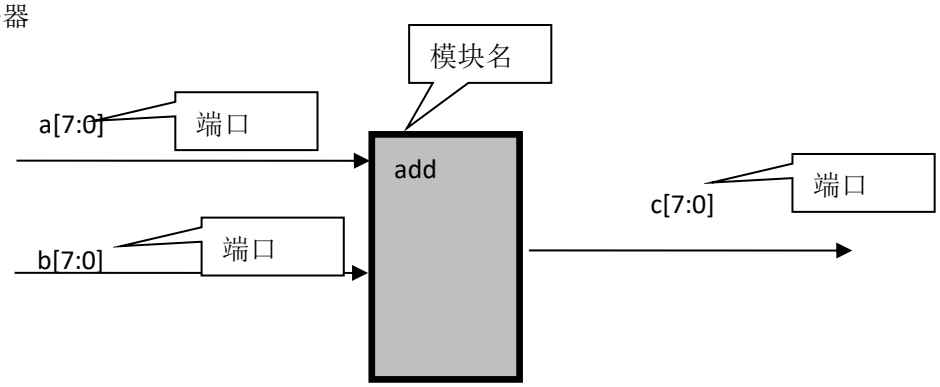
■ 信号

定义：信号本身是电路的关键属性，是电路核心，在电路设计中，信号有驱动信号，控制信号，负载信号，从模块角度来看，信号有内部信号和端口信号。

由于信号上传递信息，实际上是数，所以声明信号的时候需要指定该信号上传递的数据类型，规定数据必须是逻辑数据，verilog 中信号数据常用的如下：

| 信号类型 | 描述 |
| --- | --- |
| **reg** | 寄存器类型，　　reg 类型！ = register |
| **wire** | 线网类型，　　默认的信号类型 |

如：　8 位加法器




---

verilog　描述如下：

```
module      add
(
            wire                默认 wire
                input           [7:0]       a,
                input       wire[7:0]       b,
               output       reg [7:0]       c
);
                                reg
```

```
wire    [7:0]       c0_tmp;//定义内部信号
reg     [7:0]       c1_tmp;//定义内部信号
```

endmodule

解释：模块 ADD　定义了 6 个信号

■　电路运行特征

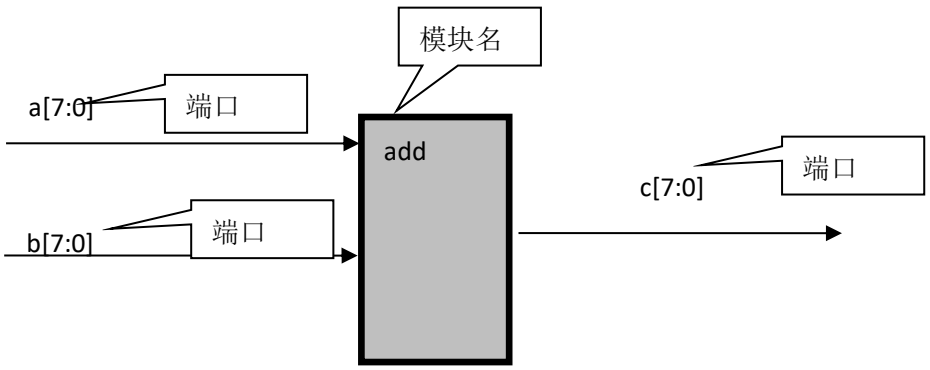
定义：当我们定义一个模块，这个模块电路如何运行需要表述出来，运行特征概括起来为 3 方面。

| 运行特征 | 描述 |
| --- | --- |
| 信号逻辑关系 | 所有信号之间的逻辑关系。 |
| 电路类型 | 我们需当前设计是组合电路还是时序，还是二者都有 |
| 初始化 | 信号上电初值，时序电路依赖于复位信号，组合通常不需要初始化 |

在 verlog 中有 3 个关键字是用来描述电路的运行的特征

| 运行特征描述关键字 | 描述 |
| --- | --- |
| **assign** | 用于信号传递的简单描述 |
| **always** | 用于描述电路类型以及信号逻辑关系的通用控制字 |
| **initial** | 用于信号初始化特征描述 |

如：　8 位加法器



verilog 描述如下：

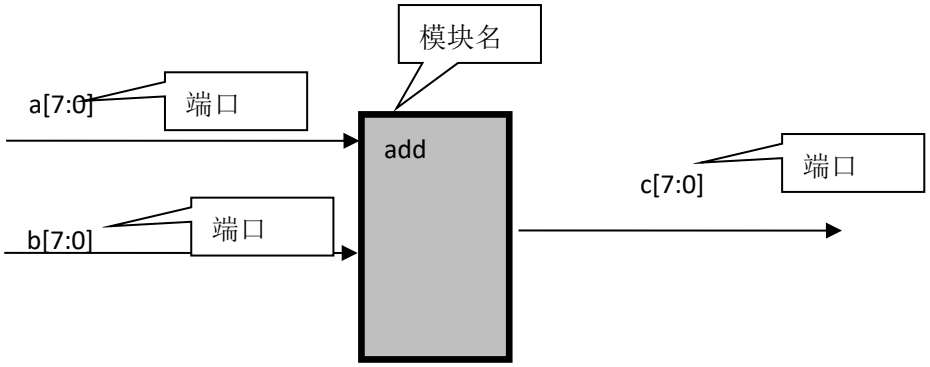
```
module        add

(
                input          [7:0]        a,
                input       wire [7:0]      b,
                 output      reg [7:0]      c
);

wire     [7:0]        c0_tmp;//定义内部信号
reg     [7:0]         c1_tmp;//定义内部信号

//通过 assiig 描述信号之间逻辑关系

assign                    c0_tmp     =   a     +      b;

//通过 always  描述组合电路以及信号之间逻辑关系
always                    @(*)
                          c1_tmp     =      a    +b;


//通过 initial    初始化
initial     c1_tmp     =   0;


assign     c    =      c0_tmp;


endmodule
```

Note:**verilog 中规定 always  iniitla 中的信号类型必须是 reg  ,assig 中的信号类型必须是 wire**。

■ 电路配置

在数字电路设计中，我们对模块设计最求更便利，具备一定的通用性，也就是模块可配置。

如：  8 位加法器




---

verilog 描述如下：

```
module        add
#(
                    parameter          WDTH    =     8


)
(
                  input            [WDTH-1:0]          a,
                  input        wire [WDTH-1:0]          b,
                   output       reg [WDTH-1:0]          c
);

wire     [WDTH:0]        c0_tmp;//定义内部信号
reg     [WDTH:0]         c1_tmp;//定义内部信号

//通过 assiig 描述信号之间逻辑关系

assign                      c0_tmp     =   a     +      b;

//通过 always  描述组合电路以及信号之间逻辑关系
always                     @(*)
                           c1_tmp     =      a    +b；


//通过 initial    初始化
initial     c1_tmp     =    0;


assign     c    =      c0_tmp;
```

endmodule

## 2.2.2HDL 文件类型

verilog 的文件后缀为*.v，按照设计文件分为以下几种类型：

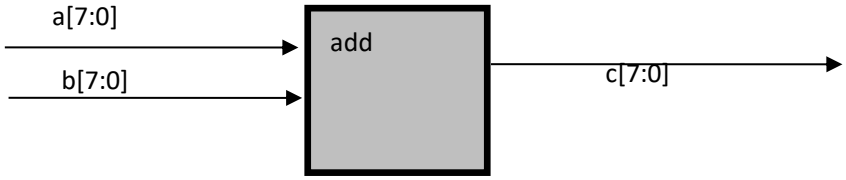
1：设计文件　　：电路设计的所有.v 文件，要求必须可综合

2：仿真辅助文件 ：用于仿真辅助的程序模块

3：tesebench　　:仿真激励文件

这里我们重点介绍 tesebench ,叫做仿真激励，用来虚拟验证设计电路的逻辑时序是否正确。

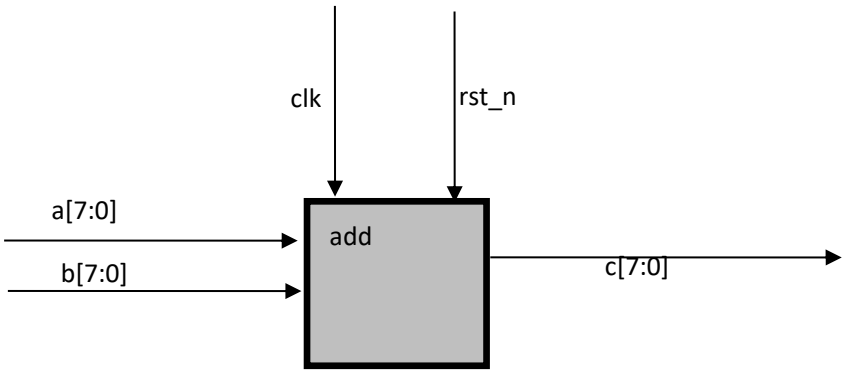
tesebench 是数字电路设计必须掌握的，为激励文件，所谓的激励就是对于一个电路系统来说，有激励和响应，针对电路系统激励信号给出的一个测试值的文件叫做激励文件，即 tesebench，在仿真中给所有激励信号一组值，我们叫做一组测试向量，激励信号多有值组成的测试向量，我们叫做完备的测试向量。




激励信号：a[7:0]，b[7:0]

响信信号　:c[7:0]

语法与 testbench 设计案例
标题：加法运算电路
要求：
1：时序电路
2：通过 tesebench 验证




## 2.2.2 HDL 语言的计算机语言特性

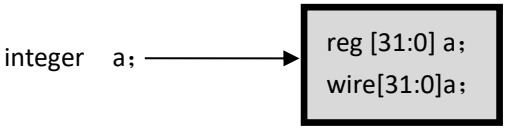
HDL 语言作为硬件设计语言，兼顾 HDL 特性和计算机语言特性的双重属性。

■ 变量

所有纯计算机机语言中变量是关键属性，用来保存数据，保存数据的存储器地址通常叫做指针，变量用来保存数据，数据需要知名类型。verlilog 中变量的定义和其他纯计算机语言一样。
如：

```
integer   a;//定义一个变量，它保存的数据为整数
real      b;//定义了一个变量，它保存的数据为实数。
```

在 verilog 中变量的定义如例子所示，他们之间也存在一定关联，某些变量可以转化为信号，比如 integer.




如果变量的值可以随机变化，通常我们把这种变量叫做变量，把值不可以变化的变量叫做常量，在 verilog 中如果变量的值恒定，即常量，我们需要单独声明。

```
parameter      COL    =   200;//定义一个全局常量，它的值恒定为 200
localparam     COL    =   200;//定义一个内部常量，它的值恒定为 200
```

使用规则如下：

```
module
#(     parameter WDDTH =8)
(...);
lacalparam ....;
....

endmodule
```

```
module
(...);
parameter    ...;
....

endmodule
```

```
module
(...);
localparam    ...;
....

endmodule
```

1:如果一个模块中同时存在模块配置定义与内部常量的定义，模块配置用 parameter ,内部量定义用 localparam.

2:如果只是模块内部常量的定义，可以用 parameter 定义

3：如果只是模块内部常量的定义，可以用 localparam 定义

■ 运算符

逻辑运算符

| 运算符 | 功能定义 | 举例 |
|---|---|---|
| & | 位与 | a[7:0] & b[7:0] =<br>a[0] & b[0]<br>a[1] & b[1]<br>a[2] & b[2]<br>a[3] & b[3]<br>a[2] & b[2]<br>a[1] & b[1]<br>a[0] & b[0] |

| | | |
|---|---|---|
| && | 逻辑与 | a[7:0] && b[7:0]<br>如<br>a=0,b=0;a[7:0] && b[7:0] = 0<br>a=2,b=0;a[7:0] && b[7:0] = 0<br>.... |
| \| | 位或 | a[7:0] \| b[7:0] =<br>a[0] \| b[0]<br>a[1] \| b[1]<br>a[2] \| b[2]<br>a[3] \| b[3]<br>a[2] \| b[2]<br>a[1] \| b[1]<br>a[0] \| b[0] |
| \|\| | 逻辑或 | a[7:0] \|\| b[7:0]<br>如<br>a=0,b=0;a[7:0] && b[7:0] = 0<br>a=2,b=0;a[7:0] && b[7:0] = 1 |
| ~ | 位取反 | ~a[7:0]=<br>~a[7]<br>~a[6]<br>...<br>~a[0] |
| ! | 逻辑非 | 如 a!= 0，表示 a 不等于 0 |
| {} | 位拼接 | {a[7:0],b[7:0]} =c[15:0] |
| << | 逻辑左移 | c=a[7:0]<<1; |
| >> | 逻辑右移 | c=a[7:0]>>1; |

数学运算

| 运算符 | 功能定义 | 举例 |
|---|---|---|
| + | 2 个数相加 | c= a[7:0] + b[7:0] ; |
| - | 2 个数相减 | c= a[7:0] - b[7:0] ; |
| * | 2 个数相乘 | c= a[7:0] * b[7:0] ; |
| / | 2 个数相除 | c= a[7:0] / b[7:0] ; |
| % | 取余 | c=a%2; |
| + | 正数 | c=+a; |

<table>
<tr><td>-</td><td>负数</td><td>c=-a;</td></tr>
<tr><td><</td><td>比较，小于</td><td>if(a<b)</td></tr>
<tr><td><=</td><td>比较，小于等于</td><td>if(a<=b)</td></tr>
<tr><td>></td><td>比较，大于</td><td>if(a>b)</td></tr>
<tr><td>>=</td><td>比较，大于等于</td><td>if(a>=b)</td></tr>
<tr><td>==</td><td>比较，等于</td><td>在 HDL 设计中数的比较中，参与比较的数值有的时候为逻辑值，逻辑值通常有 0,1，x，z。<br><br>规定该运算符不能比较 x 和 z
<table>
<tr><td>a[7:0]</td><td>b[7:0]</td><td>比较结果</td></tr>
<tr><td>8’d8</td><td>8’b8</td><td>相等</td></tr>
<tr><td>8’b1010101x</td><td>8’b10101010</td><td>无法比较</td></tr>
<tr><td>8’b1010101z</td><td>8’b10101010</td><td>无法比较</td></tr>
</table></td></tr>
<tr><td>===</td><td>比较，全等于</td><td>在 HDL 设计中数的比较中，参与比较的数值有的时候为逻辑值，逻辑值通常有 0,1，x，z。<br><br>规定该运算符可以比较 0,1，x,z；
<table>
<tr><td>a[7:0]</td><td>b[7:0]</td><td>比较结果</td></tr>
<tr><td>8’d8</td><td>8’b8</td><td>相等</td></tr>
<tr><td>8’b1010101x</td><td>8’b10101010</td><td>不相等</td></tr>
<tr><td>8’b1010101z</td><td>8’b10101010</td><td>不相等</td></tr>
</table></td></tr>
<tr><td>!=</td><td>比较，不等于</td><td>if(a!=5)</td></tr>
</table>

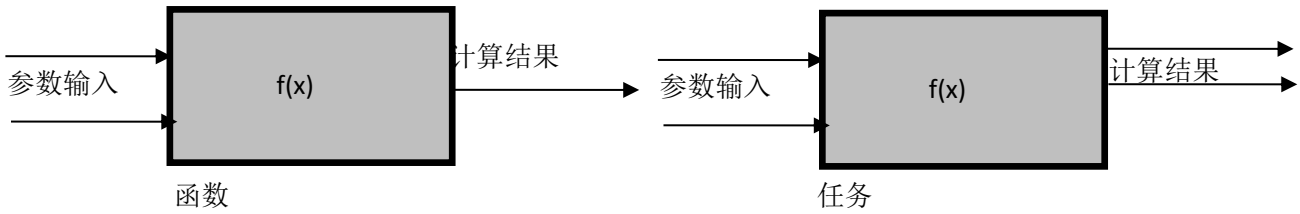
特殊标记符

在 verilog 除了普通的运算符，还有一种特殊的标记符，在运算中用来标记，突出某一个某一属性。

| 标记符 | 应用场景 |
| --- | --- |
| # | 1：标记模块的参数定义<br>add<br>#(<br>.WDTH(8)<br>)<br>add ins<br>2：延迟标记<br>#100 ; |
| ` | 表示该字不是普通的关键字，是编译指令<br>`definbe |
| ‘ | 赋值属性<br>8’d2;//突出该数为 10 进制数<br>8’h2;//突出该数为 16 进制数 |
| @ | 条件属性<br>@（*）<br><br>@（posedge clk ） |
| $ | 系统属性<br><br>$ramdom |

■ 函数，任务

在计算机语言中函数是一大特色，本质通过运算返回结果，通常函数只返回一个结果，有的时候我们希望返回多个结果，这样就衍生出任务。



关于函数和任务区别

| 区别 | 函数 | 任务 |
| --- | --- | --- |
| 参数 | 1：必须有<br>2：允许多个参数<br>3：参数是信号和变量皆可 | 1：参数有无皆可<br>2：参数是信号和变量皆可<br>3：允许多个参数 |
| 结算结果 | 1：必须有<br>2：只能是 1 个结果 | 1：有无皆可<br>2：允许返回多个结果 |

➢ 函数标准定义

```
function      [返回值位宽-1:0]      函数名；
参数声明；
局部变量的声明；
begin



    end
    endfunction
```

函数设计案例

标题：通过函数运算

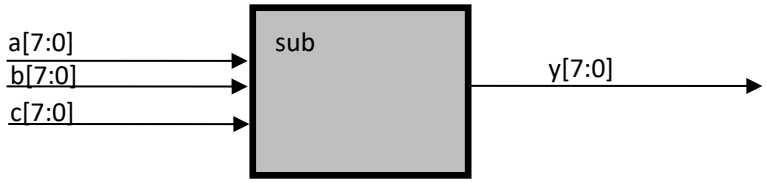
要求：

1：计算公式

y=  (a+b)  -c；

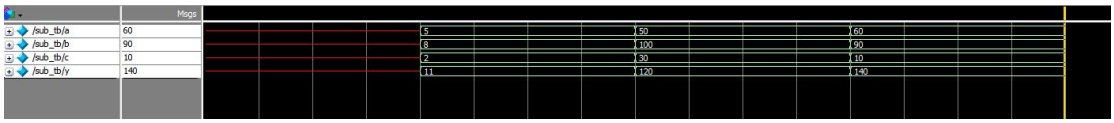
2：通过函数调用

3:a[7:0], b[7:0],c[7:0]




端口说明

| 端口信号名 | 描述 |
| --- | --- |
| 减法器参数输入 | |
| a[7:0] | 加法器参数 a[7:0] |
| b[7:0] | 加法器参数 b[7:0] |
| c[7:0] | 加法器参数 c[7:0] |
| 减法器输出结果 | |
| y[7:0] | 减法器运算结果输出 |




➢ 任务的标准定义

**task** 任务名；

参数声明；

任务局部变量声明

begin

任务内部实现；

endtask

任务设计案例

标题：通过任务运算

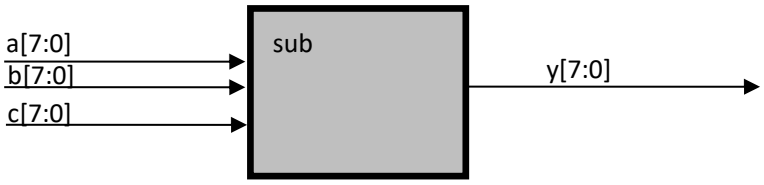
要求：

1：计算公式

y= (a+b) -c；

2：通过函数调用

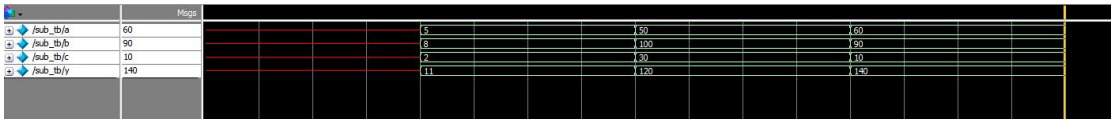
3:a[7:0], b[7:0],c[7:0]




端口说明

| 端口信号名 | 描述 |
| --- | --- |
| 减法器参数输入 | |
| a[7:0] | 加法器参数 a[7:0] |

| | |
|---|---|
| b[7:0] | 加法器参数 b[7:0] |
| c[7:0] | 加法器参数 c[7:0] |
| 减法器输出结果 | |
| y[7:0] | 减法器运算结果输出 |




任务设计案例

标题：通过任务运算

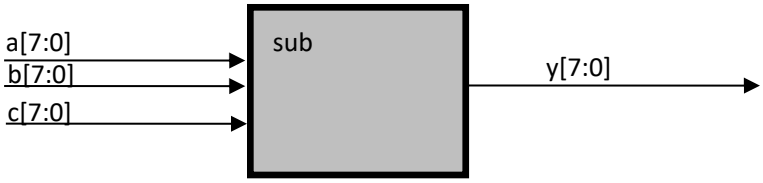
要求：

1：计算公式

y0= (a+b) -c；

y1= (a+b) +c；

y= y1 + y0;

2：y0 调用函数，y1 调用任务

3:a[7:0], b[7:0],c[7:0]